With this configuration in place, you don't have to worry about accidentally overwriting your production data during testing.

Now, it's time to think about the structure of your data. Each expense item will need several pieces of information:

- A unique ID
- Name of the payee
- Amount
- Date

You'll use a Sequel *migration* to create the table structure that holds this information.[4] You can put migration files anywhere, but a common convention is to keep them in db/migrations. Add the following code to db/migrations/0001_create_expenses.rb:

06-integration-specs/03/expense_tracker/db/migrations/0001_create_expenses.rb

```
Sequel.migration do
  change do
    create_table :expenses do
      primary_key :id
      String :payee
      Float :amount
      Date :date
    end
  end
end
```

To apply this migration to your database, you'll need to tell Sequel to run it. In a minute, you'll configure RSpec to do so automatically every time you run your integration tests. For now, try running this migration against the development database. To do so, you can use the sequel command that ships with the Sequel library:

```
$ bundle exec sequel -m ./db/migrations sqlite://db/development.db --echo
« truncated »
I, [2017-06-13T13:34:25.536511 #14630]  INFO -- : Finished applying
migration version 1, direction: up, took 0.001514 seconds
```

Now that you've configured Sequel, it's time to write some specs.

Testing Ledger Behavior

We've seen how to run Sequel migrations manually from the command line. You'll need to configure RSpec to run them automatically, so that the database structure is in place before your first integration spec runs.

4. http://sequel.jeremyevans.net/rdoc/files/doc/migration_rdoc.html